



Mongo DB

Agenda

- ▶ **Introduction to MongoDB**
- ▶ **MongoDB Structure**
- ▶ **MongoDB Commands**



Introduction to MongoDB

NOSQL

- ▶ **NOSQL – Not Only SQL - databases that store data in flexible, non-tabular forms.**
- ▶ **Supports scalability, flexibility, and high performance.**

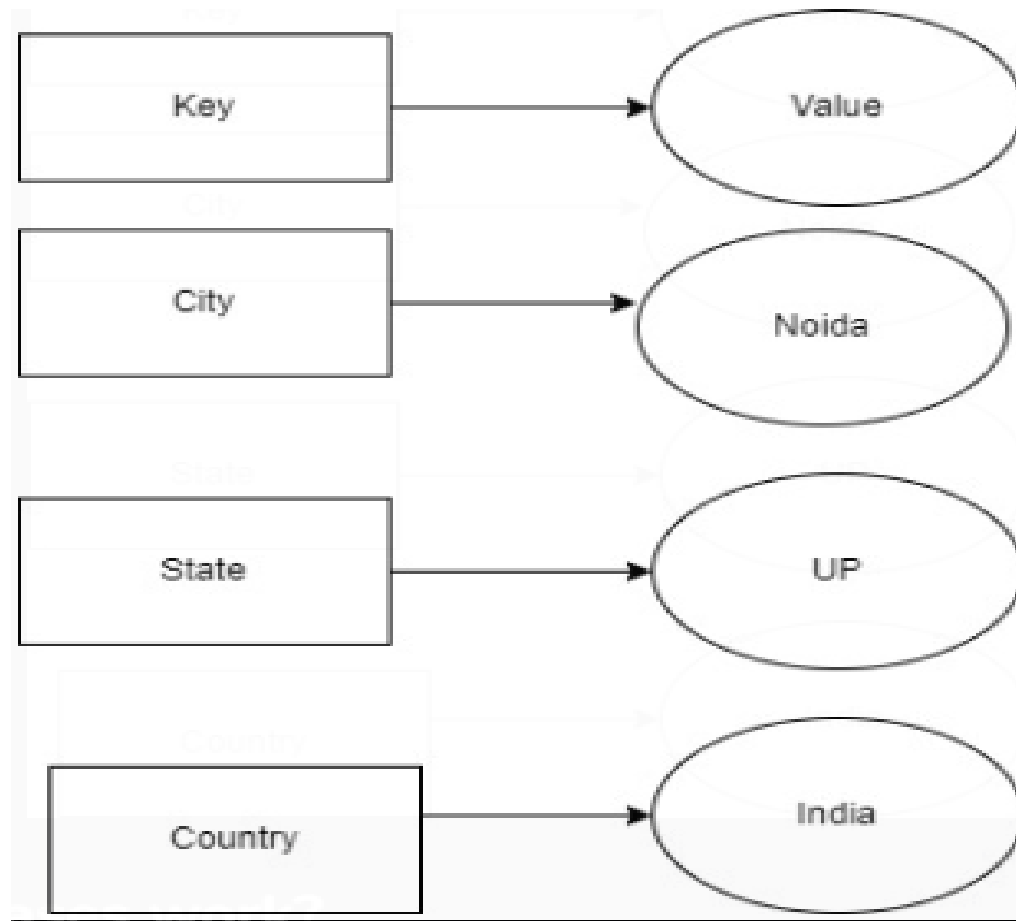
NOSQL

- ▶ **Traditional RDBMS (MySQL, Oracle, etc.) store data in tables with fixed schemas.**
- ▶ **As data grows, scaling RDBMS becomes difficult.**
- ▶ **Modern applications (social media, IoT, analytics) deal with unstructured or semi-structured data.**

Types of NOSQL

Type	Description	Example
Key-Value	Data stored as key-value pairs	Redis, DynamoDB
Document	JSON-like documents	MongoDB, CouchDB
Column	Data stored by columns	Cassandra, HBase
Graph	Nodes and edges represent relationships	Neo4j

Types of NOSQL – Key Value



Types of NOSQL - Document

```
{
  "_id":
  ObjectId("5ef2d4b45b7f11b6d7a"),
  "user_id": "Sherlock Holmes",
  "age": 40,
  "address":
    {
      "Country": "England"
      "City": "London",
      "Street": "221B Baker St."
    },
  "Hobbies": [ violin, crime-solving ]
}
```

```
{
  "_id":
  ObjectId("6ef8d4b32c9f12b6d4a"),
  "user_id": "John Watson",
  "age": 45,
  "address":
    {
      "Country": "England"
      "City": "London",
      "Street": "221B Baker St."
    },
  "Medical license": "Active"
}
```

Source: MongoDB's official slide decks

Types of NOSQL – Column

Row-oriented

ID	Name	Grade	GPA
001	John	Senior	4.00
002	Karen	Freshman	3.67
003	Bill	Junior	3.33

Column-oriented

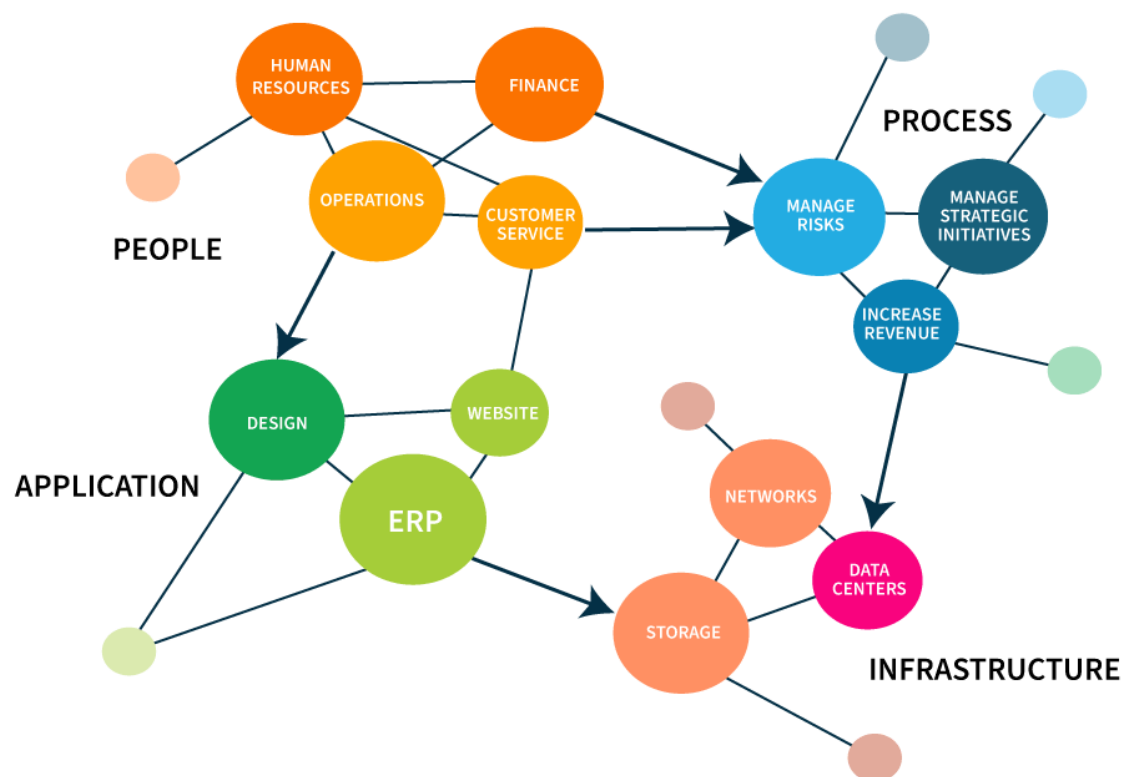
Name	ID
John	001
Karen	002
Bill	003

Grade	ID
Senior	001
Freshman	002
Junior	003

GPA	ID
4.00	001
3.67	002
3.33	003

Types of NOSQL – Graph

Graph Database



MongoDB

MongoDB

- ▶ MongoDB works as a NoSQL, document-oriented database that stores data in a flexible, JSON-like format called BSON (Binary JSON).
- ▶ Instead of rows and columns (like SQL), it uses collections and documents.

MongoDB

- ▶ **Database:** A container for collections (similar to schema in SQL).
- ▶ **Collection:** A group of documents (similar to a table).
- ▶ **Document:** A record stored in JSON-like key-value pairs

Database

Collections

Documents

Documents

Documents

Collections

Documents

Documents

Documents

MongoDB

Component	Equivalent in RDBMS	Description
Database	Database	Stores multiple collections.
Collection	Table	Groups related documents.
Document	Row	A BSON object containing key-value pairs.
Field	Column	Stores data attributes within documents.

Binary JSON

- ▶ MongoDB uses BSON, an extended version of JSON
- ▶ It supports additional data types like binary data, dates, and nested arrays.
- ▶ Improving query efficiency and storage performance.

```
{  
  "_id": ObjectId(  
    "5f4f7fef2d4b45b7f11b6d7a"),  
  "user_id": "Sean",  
  "age": 29,  
  "Status": "A"  
}
```

The Document Model: Structure and Syntax

To the left is an example of a document representing a user details including user_id, age, and a status category.

Source: MongoDB's official slide decks

```
{  
  "_id": ObjectId(  
    "5f4f7fef2d4b45b7f11b6d7a"),  
  "user_id": "Sean",  
  "age": 29,  
  "Status": "A"  
}
```

The Document Model: Structure and Syntax

A document in MongoDB uses the JavaScript Object Notation (JSON) format.

This format uses curly brackets to mark the start and the end of the document.

Source: MongoDB's official slide decks


```
{  
  "_id": ObjectId(  
    "5f4f7fef2d4b45b7f11b6d7a"),  
  "user_id": "Sean",  
  "age": 29,  
  "Status": "A"  
}
```

The Document Model: Structure and Syntax

MongoDB refers to keys
as fields.

The field-value pairs in a
document are separated by
colons (:)

Source: MongoDB's official slide decks

```
{
  "_id": ObjectId(
    "5f4f7fef2d4b45b7f11b6d7a"),
  "user_id": "Sean",
  "age": 29,
  "Status": "A"
}
```

The Document Model: Structure and Syntax

Each field must be enclosed
within quotation marks.
String values are often
quoted as good practice.

Source: MongoDB's official slide decks

```
{  
  "_id": ObjectId(  
    "5f4f7fef2d4b45b7f11b6d7a"),  
  "user_id": "Sean",  
  "age": 29,  
  "Status": "A"  
}
```

The Document Model: Structure and Syntax

Each field-value pair is
separated within the
document by commas.

Source: MongoDB's official slide decks

Querying Language

- ▶ MongoDB supports powerful indexing and real-time queries, ensuring faster read/write operations.
- ▶ Developers can store, retrieve, and update data using MongoDB Query Language (MQL), which is similar to JSON queries.

MongoDB

Component	Equivalent in RDBMS	Description
Database	Database	Stores multiple collections.
Collection	Table	Groups related documents.
Document	Row	A BSON object containing key-value pairs.
Field	Column	Stores data attributes within documents.

DOCUMENT MODEL CONSTRUCTS

- ▶ The document model representation is based on three constructs:
 - ▶ **Fields** (also referred to as **attributes**)
 - ▶ **Sub-documents** (also referred to as **nested documents** or **objects**)
 - ▶ **Arrays**



FIELDS/ATTRIBUTES

- ▶ In the Relational data model, a table would have a list of column names, which represent the table's **attributes**, and each record (i.e. **tuple**) would have a set of values in the same order as the column names.
- ▶ In MongoDB, using the document model, these column names correspond to the **field** names, whereas the relational database tuples correspond to the values of these field key-pairs.

Cars				
_id	owner	make	model	year
007	Daniel	Ferrari	GTS	1982
Q08	Daniel	Fiat	500	2013

```
{  
  "_id": 007,  
  "owner": "Daniel",  
  "make": "Ferrari",  
  "model": "GTS",  
  "year": 1982  
}
```

```
{  
  "_id": 008,  
  "owner": "Daniel",  
  "make": "Fiat",  
  "model": "500",  
  "submodel": "S",  
  "year": 2013  
}
```

FIELDS/ATTRIBUTES

- ▶ The main difference is that the attributes are appearing in each document, allowing for greater flexibility in having documents with different shapes; in other words, having different fields.
- ▶ Each document is self-describing without the need to refer to global metadata like a table definition

```
{  
  "_id": 007,  
  "owner": "Daniel",  
  "make": "Ferrari",  
  "model": "GTS",  
  "year": 1982  
}
```

```
{  
  "_id": 008,  
  "owner": "Daniel",  
  "make": "Fiat",  
  "model": "500",  
  "submodel": "S",  
  "year": 2013  
}
```

SUB-DOCUMENTS/OBJECTS

- ▶ **An *object*, or *sub-document*, in the document model allows for information to be grouped together or embedded as a one-to-one relationship between what would be two tables from the tabular relational model.**

SUB-DOCUMENTS/OBJECTS

```
1 {
2   "_id": "post_id_1",
3   "title": "Embedded vs. Referenced Documents in MongoDB",
4   "content": "MongoDB is an open-source , non-relational ,
5               NoSQL...",
6   "author": "Ashish Kumari",
7   "publishDate": "2024-06-05",
8   "comments": [
9     {
10      "commentId": "comment_id_1",
11      "author": "User 1",
12      "content": "Great post! Very informative.",
13      "date": "2024-06-09"
14    },
15    {
16      "commentId": "comment_id_2",
17      "author": "User 2",
18      "content": "I learned a lot from this article.",
19      "date": "2024-06-13"
20    }
21  ]
22 }
```

Embedded document



ARRAYS

- ▶ A list of values that is enclosed in square brackets within a document represents a field whose value is an **array**
- ▶ They represent the denormalized pattern of the relational database, which means that the “wheels” collection in the given table below, may not necessarily need a separate document, but can be embedded in the cars document as an array

Cars		
_id	owner	make
007	Daniel	Ferrari
Q08	Daniel	Fiat

Wheels	
_id	car_id
234819	007
281928	007
392838	007
928038	007
950555	008

```
{
  "_id": 007,
  "owner": "Daniel",
  "make": "Ferrari",
  "wheels": [
    { "partNo": 234819 },
    { "partNo": 281928 },
    { "partNo": 392838 },
    { "partNo": 928038 }
  ],
  ...
}
```




MQL

MongoDB QUERY LANGUAGE

- ▶ The *MongoDB Query Language* or *MQL*, was designed to have a simple syntax as well as being explicitly designed for querying documents.
- ▶ It only queries a single collection
- ▶ When you need to query multiple collections or perform more complex data processing, the MongoDB Aggregation Framework should be used.
- ▶ MQL is typically used for creating, reading, updating, or deletion (CRUD) operations.

MongoDB QUERY LANGUAGE

- ▶ **MQL's query operators help enhance the MQL CRUD operations.**
- ▶ **MQL operators vary from comparison to logical to array, as well as specific purpose operators like bitwise or geospatial. These allow for complex operations to be performed on the documents.**

MongoDB QUERY LANGUAGE

- ▶ **Some of these operators include:**
 - ▶ **Comparison**
 - ▶ **Logical**
 - ▶ **Element**
 - ▶ **Array**
 - ▶ **Evaluation**
 - ▶ **Bitwise**
 - ▶ **Comment**
 - ▶ **Geospatial**
 - ▶ **Projection, Update and Update Modifiers**

MongoDB COMMANDS

- ▶ The command `show dbs;` is used to show all available databases.

```
> show dbs
< sample_airbnb      52.82 MiB
  sample_analytics   9.16 MiB
  sample_geospatial  1.37 MiB
  sample_guides       40.00 KiB
  sample_mflix        50.33 MiB
  sample_restaurants  7.04 MiB
  sample_supplies     1.13 MiB
  sample_training     52.30 MiB
  sample_weatherdata  3.05 MiB
  admin               280.00 KiB
  local               1.68 GiB
Atlas atlas-hggiwo-shard-0 [primary] test>
```


MongoDB COMMANDS

- ▶ In order to use a database, use `use <database>;` command.
- ▶ TIP: If the database does not exist, it will be created.

```
> _MONGOSH
```

```
> use sample_restaurants
```

```
< 'switched to db sample_restaurants'
```

- ▶ To show collections in a database, use `show collections;` command.

```
> show collections
```

```
< neighborhoods
```

```
restaurants
```

```
Atlas atlas-hggiwo-shard-0 [primary] sample_restaurants>
```


MongoDB COMMANDS

- ▶ One of the ways to create a collection is to insert a document into a collection, as shown in the following example:

```
db.classical.insertOne(  
    {"name": "Sultan" }  
);
```

- ▶ Another command to create a collection is:

```
db.createCollection("classical");
```

MongoDB COMMANDS

```
> show collections
< neighborhoods
   restaurants
> db.classical.insertOne({"name" : "Sultan"})
< { acknowledged: true,
    insertedId: ObjectId("635ac8ff076fbc85d789c734") }
> show collections
< classical
   neighborhoods
   restaurants
Atlas atlas-hggiwo-shard-0 [primary] sample_restaurants > |
```

MongoDB COMMANDS: `find()`

- ▶ The `find()` command is used to return documents satisfying given search criteria.
- ▶ To find all documents in a collection, use `db.<collection>.find({})`;
- ▶ This command may return thousands of document (if in the collection).
- ▶ In order to return the first document in the collection, use `db.<collection>.findOne({})`;

MongoDB COMMANDS: find()

```
> db.restaurants.find({})
< { _id: ObjectId("5eb3d668b31de5d588f4292a"),
  address:
    { building: '2780',
      coord: [ -73.98241999999999, 40.579505 ],
      street: 'Stillwell Avenue',
      zipcode: '11224' },
  borough: 'Brooklyn',
  cuisine: 'American',
  grades:
    [ { date: 2014-06-10T00:00:00.000Z, grade: 'A', score: 5 },
      { date: 2013-06-05T00:00:00.000Z, grade: 'A', score: 7 },
      { date: 2012-04-13T00:00:00.000Z, grade: 'A', score: 12 },
      { date: 2011-10-12T00:00:00.000Z, grade: 'A', score: 12 } ],
  name: 'Riviera Caterer',
  restaurant_id: '40356018' }
{ _id: ObjectId("5eb3d668b31de5d588f4292b"),
```

MongoDB COMMANDS: find()

- ▶ We can find a document based on the position. For example, to find the third document in the collection, we can use the skip function:
 - ▶ `db.<collection>.find({}).skip(2);`
- ▶ This will display all the documents from the third one

MongoDB COMMANDS: `find()`

- ▶ If we want to restrict it to just one document, we can use `limit` function:
 - ▶ `db.<collection>.find({}).skip(2).limit(1);`
- ▶ This will display only the third document
- ▶ Can use any number inside limit to restrict the number of documents to be found – any number beyond the size of document will be assumed as “until the end”

find() QUERIES: FINDING A DOCUMENT

- ▶ In order to find a document/documents with particular values for specific fields, the find command is used with {<field> : <value>, ...} as a parameter.
- ▶ For example, in order to find all restaurants with Asian Cuisine, we formulate our query as:

```
db.restaurants.find({ "cuisine" :  
                      "Asian" });
```
- ▶ This may potentially return a large number of restaurants.

`find()` QUERIES: FINDING A DOCUMENT

- ▶ To get a count of these restaurants (serving Asian cuisine) use

```
db.restaurants.find({ "cuisine" :  
    "Asian" }).count();
```

find() QUERIES: FINDING A DOCUMENT

- ▶ For example, in order to find all restaurants with Asian Cuisine located in “Queens” borough, we formulate our query as:

```
db.restaurants.find({ “cuisine” :  
    “Asian” , “borough” : “Queens” })
```

- ▶ Can also use connectives like *and* and *or*
- ▶ The above query with “and” can be rewritten as:

```
db.restaurants.find({$and:[{ “cuisine” :  
    “Asian”}, {“borough” : “Queens” }]}))
```

`find()` QUERIES: FINDING A DOCUMENT

- ▶ Comparison operators can also be used as follows:
 - ▶ Equals → `$eq`
 - ▶ Not Equals → `$ne`
 - ▶ Less than → `$lt`
 - ▶ Less than or equal → `$lte`
 - ▶ Greater than → `$gt`
 - ▶ Greater than or equal → `$gte`
 - ▶ In → `$in`
 - ▶ Not In → `$nin`

QUERIES INVOLVING COMPARISONS

- ▶ Write a MongoDB query to find the restaurants which do not prepare American cuisine and located in “Queens” borough.
- ▶ `db.restaurants.find({"cuisine" : {$ne : "American"}, "borough" : {$eq: "Queens"}});`
- ▶ Using logical connectives
- ▶ `db.restaurants.find({ $and:
 [
 {"cuisine" : {$ne : "American "}},
 {"borough" : {$eq: "Queens"}}
]
});`

`find()` QUERIES: FINDING A DOCUMENT

- ▶ Search documents if a specific field exists irrespective of the value in them
- ▶ Write a MongoDB query to find the restaurants which have borough fields defined for them.
- ▶ `db.restaurants.find({"borough" : {$exists: true}});`
- ▶ OR
- ▶ `db.restaurants.find({"borough" : {$exists: 1}});`

find() QUERIES

- ▶ You can use `find()` to return specific fields from the returned documents by using `db.<collection>.find(queries, projection, options)`
- ▶ Write a MongoDB query to display the fields `restaurant_id`, `name`, `borough` and `cuisine` for all the documents in the collection `restaurants`.
- ▶ `db.restaurants.find( {}, {"restaurant_id" : true, "name" : true, "borough" : true, "cuisine" : true})`

find() QUERIES

```
{ _id: ObjectId("5eb3d668b31de5d588f4293c"),  
  borough: 'Brooklyn',  
  cuisine: 'American',  
  name: 'The Movable Feast',  
  restaurant_id: '40361606' }  
  
{ _id: ObjectId("5eb3d668b31de5d588f4293d"),  
  borough: 'Manhattan',  
  cuisine: 'Delicatessen',  
  name: 'Bully\'S Deli',  
  restaurant_id: '40361708' }
```

Type "it" for more

find() QUERIES

- ▶ “options” include commands like sort, limit, skip, and many more
- ▶ For example:
 - ▶ `db.restaurants.find( {}, {" restaurant_id" : true, "name" : true, "borough" : true, "cuisine" : true}).sort({"name": 1});`
 - ▶ This sorts the results in ascending, alphabetical order according to “name”

find() QUERIES: RESULTS

```
> db.restaurants.find({"cuisine" : "Asian"}).count()  
< 309
```

- In all, there are 309 restaurants in the collection serving “Asian cuisine”.
- Write a query that finds all restaurants serving Asian cuisine on the Grand Street.

find() QUERIES: RESULTS

```
> db.restaurants.find({"cuisine" : "Asian"})
< { _id: ObjectId("5eb3d668b31de5d588f42b9f"),
  address:
    { building: '51',
      coord: [ -73.9787406, 40.7611474 ],
      street: 'West 52 Street',
      zipcode: '10019' },
  borough: 'Manhattan',
  cuisine: 'Asian',
  grades:
    [ { date: 2014-08-12T00:00:00.000Z, grade: 'A', s
      { date: 2013-08-27T00:00:00.000Z, grade: 'A', s
      { date: 2013-04-03T00:00:00.000Z, grade: 'B', s
      { date: 2012-09-20T00:00:00.000Z, grade: 'A', s
      { date: 2011-08-17T00:00:00.000Z, grade: 'A', s
  name: 'China Grill',
```

find() QUERIES: RESULTS

```
> db.restaurants.find({"cuisine" : "Asian", "address.street" : "Grand Street"})  
< { _id: ObjectId("5eb3d668b31de5d588f43a8a"),  
  address:  
    { building: '245',  
      coord: [ -73.994405, 40.718029 ],  
      street: 'Grand Street',  
      zipcode: '10002' },  
  borough: 'Manhattan',  
  cuisine: 'Asian',  
  grades:  
    [ { date: 2014-12-08T00:00:00.000Z, grade: 'A', score: 10 },
```

```
> db.restaurants.find({"cuisine" : "Asian", "address.street" : "Grand Street"}).count()  
< 5
```


QUERIES INVOLVING COMPARISONS

- ▶ Find all restaurants which achieved a score of greater than 100.

- ▶ `db.restaurants.find({ "grades":
 { $elemMatch:
 { "score":
 { $gt:100 }
 }
 }
 });`

- ▶ Can also use dot operator to search for it as well

- `db.restaurants.find({"grades.score":{$gt:100}});`

QUERIES INVOLVING COMPARISONS

```
> db.restaurants.find({grades : { $elemMatch:{"score":{$gt : 100}}}})
< { _id: ObjectId("5eb3d668b31de5d588f42a92"),
  address:
    { building: '65',
      coord: [ -73.9782725, 40.7624022 ],
      street: 'West 54 Street',
      zipcode: '10019' },
  borough: 'Manhattan',
  cuisine: 'American',
  grades:
    [ { date: 2014-08-22T00:00:00.000Z, grade: 'A', score: 11 },
      { date: 2014-03-28T00:00:00.000Z, grade: 'C', score: 131 },
      { date: 2013-09-25T00:00:00.000Z, grade: 'A', score: 11 } ] }
```

QUERIES INVOLVING COMPARISONS

- ▶ Dot operator can be applied to any sub-document or an array of sub-documents
- ▶ However `$elemMatch` operator works for ONLY array of sub-documents
- ▶ For example, in the given example, `$elemMatch` will work for grades but not for address

```
< { _id: ObjectId("5eb3d668b31de5d588f42a92"),  
  address:  
    { building: '65',  
      coord: [ -73.9782725, 40.7624022 ],  
      street: 'West 54 Street',  
      zipcode: '10019' },  
  borough: 'Manhattan',  
  cuisine: 'American',  
  grades:  
    [ { date: 2014-08-22T00:00:00.000Z, grade: 'A', score: 11 },  
      { date: 2014-03-28T00:00:00.000Z, grade: 'C', score: 131 },  
      { date: 2013-09-25T00:00:00.000Z, grade: 'A', score: 11 } ]  
}
```

QUERIES INVOLVING COMPARISONS

- For example, in the given example, \$elemMatch will work for grades but not for address

```
< { _id: ObjectId("5eb3d668b31de5d588f42a92"),  
  address:  
    { building: '65',  
      coord: [ -73.9782725, 40.7624022 ],  
      street: 'West 54 Street',  
      zipcode: '10019' },  
  borough: 'Manhattan',  
  cuisine: 'American',  
  grades:  
    [ { date: 2014-08-22T00:00:00.000Z, grade: 'A', score: 11 },  
      { date: 2014-03-28T00:00:00.000Z, grade: 'C', score: 131 },  
      { date: 2013-09-25T00:00:00.000Z, grade: 'A', score: 11 } ]  
}
```

QUERIES INVOLVING COMPARISONS

- ▶ Find all restaurants which achieved a score of greater than 100.
 - ▶ `db.restaurants.find({"grades":{"$elemMatch":{"score":{"$gt":100}}}});`
 - OR
 - ▶ `db.restaurants.find({"grades.score":{"$gt":100}});`
- ▶ Find all restaurants that are in building "65"
 - ▶ `db.restaurants.find({"address.building":{"$eq":"65"}});`
 - ▶ This works but if we apply `$elemMatch`, it won't return any results, as it won't find an "element" as a sub-document, i.e.
`db.restaurants.find({"address":{"$elemMatch":{"building":{"$eq":"65"}}}});`

QUERIES INVOLVING COMPARISONS

- ▶ Dot operator works on arrays of values as well using index number
- ▶ For example, in the given document, coord is an array of numbers, so we can use indices 0 and 1 with the dot operator to access them
- ▶ Example, find all restaurants whose first coordinate (i.e. at index 0) is 79.55
- ▶ `db.restaurants.find({"address.coord.0": {$eq: 79.55}});`

QUERIES INVOLVING COMPARISONS

- ▶ Write a MongoDB query to find the restaurants which do not prepare American cuisine and achieve a score more than 70 and located in the longitude less than -65.754168.
- ▶ `db.restaurants.find({"cuisine" : {$ne : "American "}, "grades.score" : {$gt: 70}, "address.coord" : {$lt : -65.754168}});`

QUERIES INVOLVING COMPARISONS

- ▶ Write a MongoDB query to find the restaurants which do not prepare American cuisine and achieve a score more than 70 and located in the longitude less than -65.754168.
- ▶ Can use logical connectives like and, or, not as well.
- ▶ For the given query, we can also use “and” as follows:
- ▶

```
db.restaurants.find({ $and:  
    [  
      {"cuisine" : {$ne : "American "}},  
      {"grades.score" : {$gt: 70}},  
      {"address.coord" : {$lt : -  
65.754168}}  
    ]  
  }  
});
```

MongoDB INSERT

- ▶ In order to insert documents in a collection we use the following two variations:
- ▶ Observe that if the “_id” field is not specified, then it is created with a default ObjectId.

- ▶ Insert One document

```
db.restaurants.insertOne({name:"Asian"});
```

- ▶ Insert Many Documents

```
db.worker.insertMany([ {_id: "W1",Ename: "John  
Smith",ProjectID: "P1",Hours: 32.5} , {_id:  
"W2",Ename: "Joyce English",ProjectID:  
"P1",Hours: 20.0} ] );
```

MongoDB UPDATE

- To update a single document we use the command:

```
db.inventory.updateOne({ item: "paper" },  
{$set: { "size.uom": "cm", status: "P"}});
```

- If no item given in the first bracket, i.e. empty it automatically refers to the first document in the collection:

```
db.inventory.updateOne({},{$set:  
{"size.uom": "cm", status: "P"}});
```

MongoDB UPDATE

- ▶ To update a many documents we use the command:

```
db.inventory.updateMany({ "qty": { $lt: 50 } },{$set: { "size.uom": "in", status: "P"}});
```

- ▶ This updates all documents having "qty" field less than 50
- ▶ If no condition given in brackets, it applies to all documents by default

```
db.inventory.updateMany({},{$set: {"size.uom": "in", status: "P"}});
```

MongoDB UPDATE OPERATORS

- ▶ Can apply the following operations:
 - ▶ Increment/Decrement → \$inc
 - ▶ Renaming a field → \$rename
 - ▶ Adding a new value to an existing set → \$push
- ▶ For eg, update the inventory by increasing the qty of all items by 5
 - ▶ `db.inventory.updateMany({},{$inc: {"qty":5}});`
- ▶ Can decrement with the same operator. Update the inventory by decreasing the qty of all items by 2
 - ▶ `db.inventory.updateMany({},{$inc: {"qty":-2}});`
- ▶ Rename the field from "qty" to "quantity" for all documents
 - ▶ `db.inventory.updateMany({},{$rename: {"qty": "quantity"}});`
- ▶ Adds a new status 'S' for all documents that have the status field
 - ▶ `db.inventory.updateMany({status:{$exists:1}},{$push: {"status": "S"}});`

MongoDB DELETE

- ▶ The delete command removes documents from the collection.
- ▶ We can do bulk delete or delete a single document using the following commands:
 - ▶ `db.inventory.deleteOne({});`
 - ▶ `db.inventory.deleteMany({});`
 - ▶ `db.inventory.deleteMany({ status : "A" });`
 - ▶ `db.inventory.deleteOne({ status: "D" });`

This command deletes the first document that matches the given condition



THANK YOU